S4 — Energy-Change Correlation Cache for Leakage Insights

OpenAMI / MeshEMS — Continuous Leakage Detection (NESL)

Allahaddje Bonheur — IoT Fellow, 10Power (in collaboration with NESL)

2026-07-22

1 Objective

Sprint **S4**: expand the leakage data model with an **energy-change correlation cache** so that, when the leakage steps up or down, the system can look back over a short window of recent energy activity and answer *"what energy event happened when the leakage changed?"* (generate / store / consume / transform). This is the data foundation the **S5 Leakage Insights Manager** consumes.

Result: S4 is complete (code). The structures compile and the step-detection → correlation path is exercised end to end by the mock leakage ramp; real energy data will tune the thresholds.

2 Architecture

Two independent streams meet in the cache: the **leakage reading** (MD0630) drives a step detector, while the **EMS energy** (`readings[0]`) is continuously recorded into a ring buffer. On a detected step, the detector queries the ring for the nearest energy snapshot.

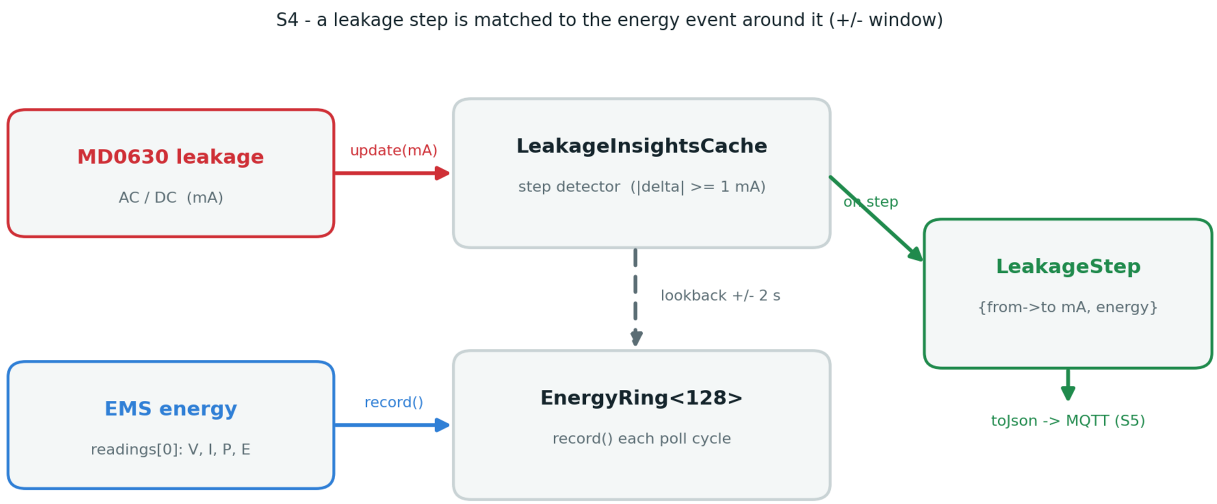

Figure 1: S4 data flow — a leakage step is matched to the energy event around it (±window).

3 Data structures

Header `include/metering/leakage_energy_correlation.h` (self-contained; reuses the `CurrentHistory` circular-buffer pattern from `data_model.h`).

Table 1: S4 types.

Type	Role
`EnergySample`	one timestamped snapshot: V, I, active/reactive power, energy, **ramp**, `phase_angle_deg` (reserved for leakage direction). Built from `PowerData`; `toJson()`.
`EnergyRing<N>`	ring of `N=128` snapshots. `record()` also derives the active-power **ramp** vs the previous sample; `lookback(ts, window)` returns the snapshot closest in time within ±window.
`LeakageStep`	a detected step: `from→to mA`, `rising`, `ts`, and the **correlated** `EnergySample`; `toJson()`.
`LeakageInsightsCache`	`update(mA, now, ring)` — flags a step when `|Δ|` `step_mA` (default **1 mA**) and grabs the nearest energy event within ±`window_ms` (default **2 s**).

4 Integration

In `src/metering/modbus_master.cpp`, globals `EnergyRing<128> energyRing` and `LeakageInsightsCache leakageInsights`. Each successful leakage poll:

1. `energyRing.record(readings[0], now)` — snapshot the EMS energy (0 on a bare bench, real once the ATM90E32 / meters run).
2. `leakageInsights.update(ac_mA, now, energyRing)` — on a step, log it with the correlated energy.

Emitted format (example, driven by the mock ramp which rises ~1.5 mA/cycle):

```
MD0630 LEAKAGE STEP UP: 28.61 -> 30.11 mA | energy corr: yes (V=230.0 I=0.00A P=0.000kW ramp=0.0
```

(V/I/P are 0 at a bare bench; they carry real values once the meters feed `readings[0]`.)

5 Design notes

- **Change-based, not level-based** — we act on *steps*, matching the standing-residual reality noted in the S5 literature review; this avoids false triggers from the always-present residual current.
- The ring is **generic** energy data, so the same pattern is reusable for other insights (neutral, harmonics) — aligning with Glenn's repeatable *Insights-Manager* framework.
- `phase_angle_deg` is carried but 0 today (the MD0630 is absolute-value only). It is exactly the field a future IVY RCM upgrade would fill for **leakage-direction** isolation (feeds S5/S6).

6 Status & next steps

- **S4 code complete** — compiles; step-detection + correlation exercised by the mock ramp.
- **Tune** `step_mA` / `window_ms` against real energy once the meters run on the bench.
- **S5** — the Leakage Insights Manager consumes `LeakageInsightsCache` and publishes correlated insights on the MQTT LV-feeder isolation subtopic (design + interactive demo already delivered).

Repository: `docs/leakage_MD0630TA1A/` — details in `S4_energy_correlation.md`; code in `include/metering/leakage_energy_correlation.h`. Companion to `S1_report.pdf`, `S2_report.pdf`; the per-sprint reports combine into the full technical document.